

SLAM-basierte Kartierung und Navigation mit dem Unitree Go2 Edu

Service- und Industrierobotik WS25/26

Prof. Dr. Anja Richert, Alexander Bauer, Leon Munz

Maschinenbau – Product Engineering and Context

Franz, Lennackers, Nothelle, Schiffmann

11154859, 11241148, 11143414, 11239930

Inhalt

Abstract	III
Symbol- und Abkürzungsverzeichnis.....	IV
1 Einleitung	5
1.1 Motivation und Kontext.....	5
1.2 Problemstellung	7
1.3 Zielsetzung	8
2 Stand der Wissenschaft und Technik.....	8
2.1 Robotische Exploration und autonome Navigation	9
2.2 Simultaneous Localization and Mapping	11
2.3 Pfadplanung und Navigation.....	12
2.4 Robotersysteme und Softwareframeworks	14
3 Aufbau des Projektes.....	16
3.1 Eingesetzte Methoden	16
3.2 Entwurfsphase.....	17
3.3 Konzeptentscheidungen	18
3.4 Technologien und Werkzeuge	20
4 Darstellung der Ergebnisse.....	21
4.1 Umgesetzte Lösung.....	22
4.2 Funktionsumfang	22
4.3 Technische Resultate	23
5 Auswertung und Diskussion der Ergebnisse	24
6 Fehlerbetrachtung und Reflexion der eigenen Vorgehensweise.....	25
6.1 Technische Limitationen	25
6.2 Methodische Einschränkungen.....	26
6.3 Ressourcenbedingte Einschränkungen	26
6.4 Learning Outcomes	26
7 Fazit.....	27

Abstract

Der vorliegende Abschlussbericht dokumentiert die Entwicklung und Evaluation eines Prototyps für die autonome Indoor-Navigation der quadrupeden Roboterplattform Unitree Go2 Education im Kontext der Objektsicherung an der Technischen Hochschule Köln. Vor dem Hintergrund des zunehmenden Fachkräftemangels im Sicherheitsgewerbe wurde untersucht, inwieweit eine kosteneffiziente Automatisierung repetitiver Kontrollrundgänge ausschließlich unter Nutzung der serienmäßig verbauten Front-LiDAR-Sensorik realisierbar ist.

Methodisch wurde hierzu eine modulare Systemarchitektur auf Basis des Robot Operating System 2 (ROS 2) implementiert und innerhalb einer Docker-Umgebung für eine reproduzierbare Software-Toolchain bereitgestellt. Der Fokus lag auf der Evaluierung des SLAM-Algorithmus KISS-ICP zur Erstellung einer zweidimensionalen Umgebungskarte (MVP 1) als notwendige Datengrundlage für eine anschließende Pfadplanung zu vordefinierten Zielpunkten (MVP 2).

Die technischen Resultate zeigen, dass die informationstechnische Anbindung sowie die Datenakquise über das Unitree-SDK erfolgreich umgesetzt werden konnten. Jedoch erwies sich der gewählte LiDAR-only-Ansatz unter Realbedingungen als instabil: Erhebliche Driften im Koordinatenursprung (Map-Frame) sowie Registrierungsfehler der Punktwolken verhinderten die Erzeugung einer konsistenten und für die Navigation verwertbaren Karte. Infolgedessen konnte die autonome Pfadplanung und Zielpunktfahrt im Rahmen des Projekts nicht validiert werden.

Die Arbeit schließt mit der Erkenntnis, dass für einen robusten Einsatz in sicherheitsrelevanten Szenarien eine eng gekoppelte Sensor-Fusion (LiDAR-IMU) oder die Integration zusätzlicher Hardware-Komponenten zwingend erforderlich ist, um die erforderliche Lokalisierungsgenauigkeit zu gewährleisten.

Symbol- und Abkürzungsverzeichnis

Abkürzung/ Formelzeichen	Beschreibung
ATE	Absolute Trajectory Error
DNN	Deep Neural Network
KISS	„Keep It Small and Simple“ (SLAM Variante)
MVPs	Minimum Viable Products
Nav2	Navigation2
ROS 2	Robot Operating System 2
RPE	Relative Pose Error
RRT	Rapidly-exploring Random Tree
RViz2	ROS 2 Visualization
SLAM	Simultaneous Localization and Mapping

1 Einleitung

1.1 Motivation und Kontext

Im Kontext öffentlicher Bildungseinrichtungen wie der Technischen Hochschule Köln spielt objektsichernde Präsenz eine zentrale Rolle für den Schutz von Personen, Infrastruktur und Sachwerten. An der Hochschule werden sicherheitsrelevante Bereiche derzeit durch manuelle Rundgänge von Sicherheitskräften abgesichert, bei denen feste Routen in Gebäuden und auf dem Außengelände abgegangen und an definierten Kontrollpunkten RFID-Checkpoints zur Dokumentation des Rundgangs gescannt werden. Dieses Vorgehen ist personalintensiv, insbesondere in Nacht-, Wochenend- und Feiertagsschichten, und geht mit körperlicher Belastung durch lange Wege sowie einer hohen Abhängigkeit von der Verfügbarkeit und Zuverlässigkeit des eingesetzten Personals einher.

Parallel dazu weist die private Sicherheitsbranche in Deutschland seit mehreren Jahren auf einen deutlichen Fachkräftemangel hin, insbesondere bei qualifiziertem und dauerhaft zuverlässigem Personal im Bereich Objekt- und Werkschutz. Branchenberichte nennen als zentrale Ursachen niedrige Lohnniveaus, unattraktive Arbeitszeiten und ein vergleichsweise geringes berufliches Ansehen, was zu Schwierigkeiten bei der Rekrutierung und Bindung geeigneter Mitarbeitender führt. Für einfache Bewachungstätigkeiten im Wach- und Sicherheitsgewerbe in Nordrhein-Westfalen liegen tarifliche Stundenlöhne typischerweise nur geringfügig über dem gesetzlichen Mindestlohn, im Bereich von etwa 14 bis 16 Euro brutto pro Stunde, und werden in der Fachberichterstattung ausdrücklich als unterdurchschnittlich attraktiv bewertet. In Kombination mit Schicht- und Nachtarbeit resultiert hieraus eine hohe Fluktuation und ein nicht unerhebliches Risiko für Qualitätsdefizite bei sicherheitsrelevanten Routinetätigkeiten [1].

Zur Einschätzung, ob die in der Literatur beschriebenen Problemlagen auch im spezifischen Kontext der TH Köln relevant sind, wurde im Vorfeld des Projekts eine explorative Kurzbefragung mit drei Mitarbeitenden aus sicherheitsnahen Bereichen (Pfortner sowie Mitarbeitende der Bibliotheks-Servicetheke) durchgeführt. Die Gespräche dienten der Gewinnung erster Einschätzungen und liefern damit eine lokale Plausibilisierung der in der Literatur beschriebenen Problemlagen.

Es ergab sich ein konsistentes Bild: Nach Einschätzung der Befragten ist geeignetes Security-Personal schwer zu finden, die Qualität der durchgeführten Kontrollen wird teilweise

als unzureichend wahrgenommen und die Bezahlung gilt als eher unattraktiv. Zusätzlich wurde ein autonomes Rundgangsystem von den Befragten als grundsätzlich sinnvolle und entlastende Unterstützung für das vorhandene Personal bewertet. Diese subjektiven Einschätzungen decken sich mit den in der Literatur beschriebenen Befunden zu Fachkräftemangel, Lohnniveau und Fluktuation im Sicherheitsgewerbe und stützen somit die Annahme, dass auch im Hochschulkontext ein struktureller Bedarf an alternativen, technologiegestützten Lösungen besteht.

Vor diesem Hintergrund adressiert das vorliegende Projekt nicht das generelle Fehlen von SLAM-Verfahren (Simultaneous Localization and Mapping) auf dem Unitree Go2, sondern eine spezifische Funktionslücke im dokumentierten Anwendungsspektrum: die fehlende, nachvollziehbar beschriebene Fähigkeit, sich auf Basis einer zuvor erstellten und gespeicherten Karte autonom in Innenräumen zu bewegen, unter Nutzung ausschließlich des im Education-System verbauten Front-LiDAR-Sensors. Bestehende Ansätze zur autonomen Navigation in sicherheitsrelevanten Szenarien setzen häufig auf zusätzliche 3D-LiDAR-Sensorik, deren Anschaffungskosten typischerweise im Bereich von etwa 3.000 bis 6.000 Euro liegen und die mit zusätzlichem Integrationsaufwand verbunden ist [2].

Ziel des Projekts ist es daher, im realen Anwendungsszenario der TH Köln eine erste Implementierung zu realisieren, bei der der Unitree Go2 mithilfe des serienmäßig verbauten Front-LiDARs eine SLAM-basierte Kartierung ausgewählter Bereiche erstellt und anschließend autonom entlang sicherheitsrelevanter Routen navigiert. Auf diese Weise soll der Roboter perspektivisch nächtliche Kontrollgänge mit Checkpoint-Erfassung unterstützen oder teilweise übernehmen, ohne dass zusätzliche, kostenintensive 3D-LiDAR-Hardware erforderlich ist. Die Kombination aus identifiziertem Fachkräftemangel, begrenzter Attraktivität der bestehenden Arbeitsbedingungen und dem Kostenvorteil eines reinen Front-LiDAR-Ansatzes motiviert somit die Entwicklung eines autonomen Security-Roboters im Hochschulkontext. Das Projekt versteht sich dabei sowohl als Beitrag zur Entlastung knapper personeller Ressourcen als auch als experimentelle Plattform, um Verfahren der SLAM-basierten Navigation unter realen Rahmenbedingungen zu erproben und hinsichtlich ihrer Praxistauglichkeit für sicherheitsrelevante Anwendungen zu bewerten [3].

1.2 Problemstellung

Wie in Abschnitt 1.1 beschrieben, erfolgt die Absicherung sicherheitsrelevanter Bereiche der TH Köln derzeit ausschließlich durch manuelle Rundgänge des Security-Personals mit RFID-basierten Checkpoints. Vor diesem Hintergrund besteht das zentrale Problem darin, dass bislang kein automatisiertes System existiert, das diese wiederkehrenden Kontrollgänge in Gebäuden und auf dem Campus technisch unterstützt oder teilweise übernehmen kann und dabei die spezifischen Rahmenbedingungen der Hochschule berücksichtigt.

Mit dem Unitree Go2 steht der Hochschule zwar eine hochgradig mobile, vierbeinige Robotik Plattform zur Verfügung, jedoch wird diese bislang vor allem für Lehr- und Demonstrationszwecke genutzt. Aus Anwendersicht fehlt eine praxistaugliche, nachvollziehbar dokumentierte Lösung, mit welcher der Roboter sicherheitsrelevante Rundgänge auf Basis einer zuvor erstellten Karte selbstständig ablaufen und dabei typische Kontrollaufgaben übernehmen kann.

Auf technischer Ebene besteht die zentrale Herausforderung darin, dass viele in der Literatur und in industriellen Anwendungen beschriebene Ansätze für autonome Sicherheitsrundgänge auf zusätzliche 3D-LiDAR-Sensorik und komplexe Sensorsuiten zurückgreifen, deren Kosten und Integrationsaufwand für den Hochschulkontext beträchtlich sind. Für den Unitree Go2 in der an der TH Köln eingesetzten Education-Variante liegt hingegen keine etablierte und detailliert beschriebene Lösung vor, die eine robuste SLAM-basierte Kartierung und anschließende autonome Navigation ausschließlich mit dem serienmäßig verbauten Front-LiDAR in typischen Innenraumumgebungen der Hochschule ermöglicht.

Vor diesem Hintergrund, lässt sich die zentrale Problemstellung wie folgt zusammenfassen: Es fehlt eine kosteneffiziente, technisch belastbare und hinreichend dokumentierte Lösung, mit welcher der Unitree Go2 auf Basis des integrierten Front-LiDAR-Sensors eigenständig SLAM-basierte Karten ausgewählter Hochschulbereiche aufbauen und diese anschließend für wiederholbare, sicherheitsrelevante Rundgänge nutzen kann, um das vorhandene Sicherheitspersonal bei routinemäßigen Kontrollaufgaben wirksam zu entlasten.

1.3 Zielsetzung

Ausgehend von der in Abschnitt 1.2 beschriebenen Problemstellung besteht die Zielsetzung dieses Projektes darin, einen prototypischen Ansatz für autonome Indoor-Navigation mit dem Unitree Go2 im Kontext der TH Köln zu entwickeln. Die langfristige Vision ist ein System, das ausgewählte Innenbereiche der Hochschule selbstständig kartiert und vordefinierte Zielpunkte entlang sicherheitsrelevanter Routen anfahren kann.

Im Rahmen des vorliegenden Semesterprojekts wurde diese Vision bewusst auf zwei klar umrissene Kernziele eingegrenzt. Erstens sollte der Roboter mithilfe des serienmäßig verbauten Front-LiDAR-Sensors in die Lage versetzt werden, eine zweidimensionale, SLAM-basierte Karte eines ausgewählten Innenbereichs der Hochschule zu erzeugen und diese in geeigneter Form zu speichern (MVP 1: „SLAM mit Front-LiDAR: Erstellung und Speicherung der Karte“). Zweitens sollte auf Basis dieser gespeicherten Karte die Berechnung kollisionsfreier Pfade zu vordefinierten Zielpunkten ermöglicht werden, um die grundlegende Eignung der erzeugten Karten für Pfadplanung und spätere Rundgänge zu untersuchen (MVP 2: „Pfadplanung auf gespeicherter Karte“).

Das Projekt versteht sich damit als Proof-of-Concept auf Funktionsebene. Ziel war nicht die vollständige Integration in den laufenden Sicherheitsbetrieb oder eine umfassende arbeitsorganisatorische Evaluation, sondern zu untersuchen, inwieweit SLAM mit dem Front-LiDAR des Unitree Go2 und eine darauf aufbauende Pfadplanung unter den typischen Rahmenbedingungen der TH Köln technisch umsetzbar sind. Weitergehende Fragestellungen sind bezogen auf Robustheit im Dauereinsatz, zur Anbindung an bestehende Leitstellenprozesse oder zu tatsächlichen Entlastungseffekten für das Sicherheitspersonal. Diese werden als Gegenstand nachgelagerter Arbeiten betrachtet.

2 Stand der Wissenschaft und Technik

Die Realisierung eines autonomen mobilen Robotersystems für sicherheitsrelevante Aufgaben erfordert die Integration verschiedener technologischer Disziplinen, die von der Sensorik über die Algorithmik bis hin zur Software-Architektur reichen. Um die theoretische Basis für das vorliegende Projekt zu schaffen, ist eine fundierte Einordnung der bestehenden Verfahren in

den Bereichen der Umgebungserfassung, der Positionsbestimmung und der autonomen Entscheidungsfindung erforderlich.

Das folgende Kapitel gibt zunächst einen Überblick über die Methoden der robotischen Exploration und die Konzepte der autonomen Navigation. Darauf aufbauend werden die Verfahren der simultanen Lokalisierung und Kartierung (SLAM) detailliert betrachtet, da diese die essenzielle Voraussetzung für die Orientierung in unbekannten Umgebungen bilden. Weiterhin werden die algorithmischen Grundlagen der Pfadplanung und lokalen Navigation erläutert, die eine kollisionsfreie Bewegung des Systems gewährleisten. Abschließend erfolgt eine Vorstellung der eingesetzten Software-Frameworks sowie der technischen Spezifikationen der genutzten Roboterplattform, um die technologische Einbettung des Vorhabens zu verdeutlichen.

2.1 Robotische Exploration und autonome Navigation

Die autonome Exploration unbekannter Umgebungen stellt eine zentrale Herausforderung in der Robotik dar, sobald diese mobil wird. Sie besteht aus der eigenständigen Umgebungserkundung und gleichzeitiger Bestimmung der eigenen Position in dieser. Das Problem tritt auf, wenn diese Umgebungen vorab nicht kartiert und deshalb Entscheidungen auf Basis sensorischer Daten in Echtzeit getroffen werden müssen. Dies ist in Szenarien wie beispielsweise Such- und Rettungseinsätzen, Sicherheitsrundgängen, industriellen Inspektionen oder Explorationsmissionen der Fall.

Die bekannteste klassische Methode zur robotischen Exploration stellt die Frontier-basierte Exploration [4], welche erstmals von Yamauchi vorgestellt wurde, dar. Bei dieser Methode wird die erstellte Kartierung in Zellen aufgeteilt und es wird jeweils eine Wahrscheinlichkeit angegeben, ob diese Zelle „belegt“, also blockiert oder „frei“ begehbar ist. Die für dieses Verfahren ausschlaggebende weitere Option stellt die Möglichkeit „unbekannt“ dar. Werden diese sogenannten „Frontiers“ erkannt, navigiert das Robotiksystem zu dieser Stelle, um Informationen zu dieser unbekannten Zelle zu finden und die Karte so schrittweise zu erschließen. Dieses Verfahren bildet bis heute durch seine robuste Funktionsweise eine grundlegende Methode für explorative Navigation mobiler Robotik.

Neben diesem Ansatz gibt es zahlreiche weitere Strategien, um die Effizienz der Exploration zu verbessern. Eine wichtige Methode stellt hierbei die sogenannte Next-Best-View-Methode dar.

Dabei generiert der Roboter mögliche Zielpositionen welche den erwarteten Informationsgewinn, sowie die dafür benötigte Bewegung (Kosten) mit einbeziehen [5]. Das Hauptaugenmerk liegt hierbei darauf, jene Position auszuwählen, von der aus möglichst viele bislang unbekannte Bereiche der Umgebung beobachtet werden können. Solche informationsbasierten Strategien werden häufig im Kontext des sogenannten Active SLAM eingesetzt, bei dem Exploration und Lokalisierung gemeinsam optimiert werden.

Durch immer weiter steigende Rechenleistungen und der Verfügbarkeit großer Datensätze gibt es seit einigen Jahren Ansätze für lern-basierte Methoden. Deep-Learning-basierte Methoden verfolgen im Gegensatz zu klassischen, regel- oder geometriebasierten Verfahren einen datengetriebenen Ansatz [6]. Sie können aus Sensordaten direkt relevante Informationen wie Pose, Tiefenschätzung oder semantische Szenenmerkmale ableiten und dadurch die Exploration stärker an inhaltlich bedeutsamen Umweltinformationen ausrichten. Ihr Potenzial liegt insbesondere in der Verknüpfung von Wahrnehmung und Interpretation, etwa durch Objekterkennung oder semantische Segmentierung. Die Semantik der Modelle lässt sich dabei in Simulationen trainieren. Somit ist ein Training in der physischen Welt vermeidbar [7].

Kritisch ist jedoch die praktische Umsetzbarkeit zu bewerten. Im Vergleich zu Frontier- oder Best-View-Methoden sind Deep Neural Network (DNN)-basierte Ansätze meist deutlich rechenintensiver, stark von Trainingsdaten abhängig und in unbekannten oder sich verändernden Umgebungen oft nur eingeschränkt robust. Zudem ist ihre Entscheidungsfindung weniger transparent und ihre Unsicherheit schwerer zu modellieren. Für den praktischen Einsatz auf mobilen Robotern sind klassische Methoden daher häufig einfacher, ressourcenschonender und besser nachvollziehbar, während Deep-Learning-Ansätze aktuell vor allem als ergänzende Erweiterung für semantisches Verständnis interessant sind, jedoch nicht in jedem Fall eine direkt besser umsetzbare Alternative darstellen.

Trotz der großen Fortschritte in diesem Forschungsfeld bestehen weiterhin zahlreiche Herausforderungen. Dazu zählen insbesondere die robuste Navigation in dynamischen Umgebungen, der Umgang mit Sensorausfällen sowie die effiziente Exploration großer oder komplex strukturierter Räume. In vielen praktischen Anwendungen wird Exploration daher eng mit Verfahren zur simultanen Lokalisierung und Kartierung (Simultaneous Localization and Mapping, SLAM) kombiniert. Diese ermöglichen es dem Roboter, während der Bewegung eine konsistente Karte der Umgebung aufzubauen und gleichzeitig seine eigene Position darin zu bestimmen.

Im Kontext dieser Arbeit liegt der Fokus auf der Exploration und Kartierung von Innenräumen mithilfe eines mobilen Robotersystems. Dabei wird insbesondere untersucht, inwieweit vorhandene Sensorsysteme eines Roboters genutzt werden können, um eine Umgebung autonom zu erfassen und eine nutzbare Karte für anschließende Navigationsaufgaben zu erzeugen. Im folgenden Abschnitt werden daher zentrale Verfahren von SLAM vorgestellt, die eine grundlegende Voraussetzung für die autonome Navigation mobiler Robotersysteme darstellen.

2.2 Simultaneous Localization and Mapping

Simultaneous Localization and Mapping (SLAM), bezeichnet das Problem, dass ein mobiler Roboter gleichzeitig eine Karte seiner Umgebung aufzeichnet sowie seine eigene Position innerhalb dieser Karte schätzt. Dies ist notwendig, da zu Beginn weder die Karte noch die eigene Position bekannt ist. SLAM bildet deshalb eine zentrale Grundlage für autonome Navigation, Exploration und nachgelagerte Planungsaufgaben da sie auf einer funktionsfähigen Lokalisierung und Kartierung aufbauen [6][8].

Die verbreitetste Variante ist dabei LiDAR-SLAM, diese nutzt Laserscanner und kann die Umgebung geometrisch erfassen. Im Vergleich zu kamerabasierten Verfahren bietet der Einsatz eines LiDARs den Vorteil, dass Entfernungen direkt gemessen werden. Zudem ist die Erfassung weniger stark von Beleuchtungsverhältnissen abhängig. Deshalb wird LiDAR-SLAM besonders häufig in Anwendungen eingesetzt, in denen robuste geometrische Karten und eine zuverlässige Navigation in Innenräumen oder bei wechselnden Lichtbedingungen erforderlich sind [8][9][10].

Aus methodischer Sicht lassen sich SLAM-Verfahren unter anderem in filterbasierte und graphbasierte Ansätze einteilen. Filterbasierte Verfahren beschreiben den Zustand bzw. die Position des Roboters mithilfe von Wahrscheinlichkeiten, also probabilistisch. Sie aktualisieren diesen rekursiv mithilfe von Vorhersage- und Korrekturschritten. Oft basieren sie auf dem Kalman-Filter oder seinen Erweiterungen. Dieses Verfahren überzeugt durch seine Echtzeitfähigkeit und den fortlaufenden Korrekturmechanismen. Deshalb sind die besonders für Odometrie- sowie Zustandsabschätzungsaufgaben geeignet [6][8]. Graphbasierte Methoden nutzen dagegen Positionen und Messdaten um Knoten (Positionen) und Kanten (Messinformationen zwischen Knoten) eines Optimierungsgraphen zu modellieren. Dadurch kann Drift

(Messverschiebung) lokal kompensiert und bei wiedererkannten Umgebungsbereichen auch global korrigiert werden (Loop-Closure). Diese Eigenschaft sorgt für die Erzeugung konsistenter Karten, über längere Trajektorien hinweg [8][10].

Ein aktuelles Beispiel für ein filterbasiertes LiDAR-IMU-Verfahren stellt FAST-LIO2 dar. Das System baut auf einem eng gekoppelten iterierten Kalman-Filter auf und implementiert direkt registrierte Rohpunkte (aus der Pointcloud) in die Karte, ohne zuvor explizite Merkmale extrahieren zu müssen. Zudem werden Beschleunigungswerte aus dem IMU ausgelesen und mit in die Kartierung implementiert, um den Drift möglichst klein zu halten. Laut Abstract sind genau diese Eigenschaften, also der iterierte Kalman-Filter und die direkte Registrierung von Rohpunkten die zentralen Neuerungen des Verfahrens. FAST-LIO2 zeichnet sich deshalb durch eine schnelle, robuste und vielseitige LiDAR-inertiale Odometrie aus [9].

Dagegen verfolgt KISS-SLAM einen LiDAR-only-Ansatz und strebt an, wie bereits im Namen beschrieben („Keep It Small and Simple“), möglichst „Out of the Box“ eine zuverlässige SLAM-Methode zu sein. Das Verfahren baut auf LiDAR-Odometrie auf, korrigiert Drift durch den Abgleich lokaler Karten und optimiert die Trajektorie in einem Pose-Graph-Optimierungsschritt. Die Autoren heben hervor, dass das System trotz geringer Komplexität wettbewerbsfähige Genauigkeit erreicht, wenig Parametrierung benötigt und konsistente Karten für nachgelagerte Aufgaben wie Navigation erzeugt [10].

Zusammenfassend zeigt sich, dass SLAM kein einzelner Algorithmus, sondern eine Verfahrensklasse mit unterschiedlichen Schwerpunkten ist. Während filterbasierte Ansätze eher für schnelle, rekursive Zustandsabschätzungen relevant sind, bieten graphbasierte Verfahren hinsichtlich der globalen Konsistenz und Driftkorrektur Vorteile. FAST-LIO2 und KISS-SLAM repräsentieren diese beide Kategorien exemplarisch und eignen sich daher gut für Einordnung moderner, LiDAR-basierter Mapping Systeme.

2.3 Pfadplanung und Navigation

Um eine autonome Fortbewegung eines mobilen Roboters gewährleisten zu können, wird neben einer Karte der jeweiligen Umgebung eine Pfadplanungs- sowie Navigationsmethode benötigt. Sie gewährleistet einen kollisionsfreien und möglichst effizienten Pfad von einer Start- zu einer Zielposition. Dies ist von der Navigation zu differenzieren, die im Gegensatz dazu die

lokale Hindernisvermeidung, Bewegungsregelung und Zielverfolgung übernimmt. In einem realen, autonomen mobilen Robotik System arbeiten diese zwei Teilaufgaben eng zusammen. Dabei bestimmt die Pfadplanung eine realisierbare Route, während die Navigation gewährleistet, dass diese Route unter realen Bedingungen robust auf Störfaktoren reagieren kann [11][12].

Klassische Ansätze der globalen Pfadplanung basieren häufig auf diskreten also für den Algorithmus vereinfachten Repräsentationen der Umgebung wie zum Beispiel Rastern (Aufteilung in Zellen) oder Graphen (Knoten und Verbindungen). Zu den wohl bekanntesten Methoden zählen Dijkstra und A*. Dijkstra bestimmt den Weg vollständig im Suchraum und wählt dabei den kostengünstigsten Suchraum. A* nutzt zusätzlich heuristische Informationen also Schätzungen über die Kosten basierend auf beispielsweise vorherigen Pfaden. Deshalb arbeitet A* unter Einbeziehung dieser Daten oft effizienter und gehört deshalb zu den am häufigsten eingesetzten Algorithmen in der mobilen Robotik [11][13].

Für komplexere oder hochdimensionale Planungsprobleme werden außerdem samplingbasierte Verfahren wie Rapidly-Exploring Random Tree (RRT oder RRT*) verwendet. Diese Methoden erzeugen stichprobenartig mögliche Zustände im Suchraum und eignen sich insbesondere für große, kontinuierliche oder nichtlinear beschränkte Umgebungen. Der Algorithmus erzeugt dabei randomisiert einen „Pfadbaum“ zum Zielpunkt. Dabei entsteht oft eine zackige Charakteristik der Pfade. RRT* unterscheidet sich dabei von RRT durch eine kontinuierliche Optimierung des Pfades. In klassischen Indoor-Navigationssystemen auf Gitterkarten, dominieren jedoch weiterhin graphbasierte Verfahren (wie Dijkstra oder A*), da sie bei bekannter Karte gut interpretierbar, robust und vergleichsweise einfach zu implementieren sind [9][11].

Um die geplanten Pfade zuverlässig umsetzen zu können, ist für die reale Anwendung die lokale Navigation nötig, da der Roboter während der Ausführung beispielsweise auf unvollständige Karten, Sensorausfälle oder plötzlich auftretende Hindernisse reagieren muss. Navigationssysteme arbeiten deshalb mit einer Kombination von globalen Planern und lokalen Reglern bzw. Planern. Diese können Hindernisse berücksichtigen und die Bewegung kontinuierlich anpassen. In ROS 2 (Robot Operating System 2) -basierten Robotersystemen wird diese Funktionalität typischerweise durch modulare Navigationsframeworks bereitgestellt. Durch diese wird Planung, Steuerung, Kosten sowie Recovery Mechanismen getrennt aber zueinander passend implementiert [12].

Ein verbreitetes Framework im ROS-2-Kontext stellt Navigation2 (Nav2) dar. Nav2 stellt eine Navigationsarchitektur bereit, die die benötigten Komponenten wie zum Beispiel globale Planung, Controller, oder die Kostenvergabemechanismen bei der Pfadplanung integriert. Damit lassen sich bestehende SLAM-Karten direkt für die Zielnavigation und damit für autonome Fahrten mobiler Robotik-Systeme nutzen [12].

Aktuell lässt sich zudem erkennen, dass sich Pfadplanung und Navigation zunehmend in Richtung hybrider und lernbasierter Ansätze entwickeln. Neben klassischen graph- und sampling-basierten Verfahren werden verstärkt Reinforcement-Learning-Methoden, Optimierungsverfahren und kombinierte Architekturen untersucht. Dennoch bleiben klassische Planer in vielen praktischen Anwendungen weiterhin relevant, da sie Transparenz und Determination aufweisen und der Integrationsaufwand vergleichsweise gering ist [11][13].

Für die vorliegende Arbeit ist insbesondere die Navigation auf einer zuvor erzeugten Karte relevant. Entscheidend ist dabei weniger die Entwicklung eines neuartigen Planungsalgorithmus als vielmehr die Frage, wie eine durch Front-LiDAR und SLAM erzeugte Karte zuverlässig für die Zielpunktfahrt und sichere Bewegung des Roboters genutzt werden kann. Daraus ergibt sich die Relevanz etablierter Navigationsframeworks und klassischer globaler Planungsverfahren für das betrachtete Robotersystem.

2.4 Robotersysteme und Softwareframeworks

Um autonome Robotik umzusetzen sind neben den zuvor vorgestellten Algorithmen auch die zugrunde liegenden Robotersysteme sowie die dazugehörigen Softwareframeworks ausschlaggebend. Sie bestimmen grundlegend, welche Sensorik verfügbar und wie Daten zwischen Systemkomponenten ausgetauscht werden. Außerdem ist der Integrationsaufwand für Funktionen wie SLAM, Pfadplanung oder Navigation maßgeblich von der Auswahl des Robotik-Systems sowie des Softwareframeworks abhängig. Im Rahmen dieser Arbeit sind insbesondere ROS2, das Robotersystem Unitree Go2 sowie die verwendete LiDAR-Sensorik relevant und wird werden im nachfolgenden Teil behandelt.

ROS 2 ist ein Middleware-framework für Robotersysteme, welches modular aufgebaut ist. Im Kern basiert es auf einem stark typisierten Publish/Subscribe-Mechanismus. Es organisiert verschiedene Softwaremodule als Netzwerk aus Nodes, welche über Topics, Services, Actions

und Parameter miteinander kommunizieren. Dadurch können verteilte und wiederverwendbare Robotik Architekturen wie Sensorverarbeitung, Zustandsabschätzung, Kartierung und Navigation modular miteinander gekoppelt werden. Gerade für komplexere mobile Robotersysteme ist diese Struktur von Vorteil, da einzelne Softwaremodule unabhängig voneinander entwickelt, getestet und ausgetauscht werden können [11].

Das verwendete Robotiksystem stellt in dieser Arbeit der Unitree Go2 dar. Der Go2 ist ein quadrupeder mobiler Roboter mit integrierter Rechenplattform und serienmäßig verbauten Front-LiDAR. Laut offizieller Produktseite beträgt das Sichtfeld des LiDARs $360^\circ \times 96^\circ$. Darüber hinaus ist in der Entwicklerdokumentation beschrieben, dass LiDAR-Punktwolken direkt auf dem System verfügbar sind und dass für erweiterte Modelle bzw. Konfigurationen (beispielsweise der Verbau eines anderen LiDARs) SLAM- und Navigationsschnittstellen direkt auf dem System verfügbar sind. Damit stellt der Go2 eine potenzielle entwicklungsfähige Plattform dar, auf der Wahrnehmung, Kartierung und Bewegung auf derselben Hardwarebasis untersucht werden können [12][13].

Um eine Integration des Go2 in robotische Softwarestacks zu gewährleisten, ist es relevant, dass Unitree, ROS 2 kompatible Schnittstellen bereitstellt. In den Dokumentationen des Roboters wird aufgeführt, dass die unteren Softwareschichten also die hardwarenahen Funktionen des Roboters über ROS 2 ansteuer- und auslesbar sind. LiDAR-Daten werden beispielsweise als ROS-Topic bereitgestellt und können so in entsprechenden Entwicklungsumgebungen wie etwa RViz2 visualisiert werden. Dadurch kann eine Brücke zwischen herstellerspezifischer Robotik Software und standardisierten ROS 2 basierten Entwicklungs- und Testmethoden aufgebaut werden [16].

Die technische Entwicklung der LiDAR-Technologie trägt maßgeblich zu der Robustheit des Autonomiegrades mobiler Roboter bei. In den letzten 25 Jahren konnte eine signifikante Entwicklung in dieser Branche beobachtet werden. Von 2D-Laserscannern mit geringer Auflösung zu 3D-Sensoren mit dichteren Punktwolken. Der 3D-LiDAR kann die Umgebung volumetrisch erfassen und ist dadurch robuster gegenüber Höhenunterschieden, Überhängen und komplexen, räumlichen Strukturen. Solche Eigenschaften sind insbesondere für globale Lokalisierung, robuste Kartierung und Navigation in komplexen Umgebungen vorteilhaft [17]. Dabei ist die Positionierung des LiDAR-Sensors zu beachten, um eine effektive Nutzung dessen gewährleisten zu können. Der verbaute Front-LiDAR des Go2 ist dabei im Sichtfeld insbesondere nach hinten, durch den eigenen Körper blockiert, eingeschränkt und liefert deshalb weniger

vollständige räumliche Informationen als nachrüstbare Alternativen auf dem Rücken des Roboters [16].

Zusammenfassend bilden ROS 2 als modulares Softwareframework, der Unitree Go2 als mobile Robotikplattform und die eingesetzte LiDAR-Sensorik die technische Grundlage des Systems. Während ROS 2 die softwareseitige Modularisierung und Integration ermöglicht, stellt der Go2 die physische Plattform mit eingebetteter Sensorik und Rechenleistung bereit. Die Wahl und Leistungsfähigkeit der LiDAR-Sensorik bestimmen wiederum maßgeblich, wie robust Kartierung, Lokalisierung und Navigation unter den gegebenen Randbedingungen umgesetzt werden können.

3 Aufbau des Projektes

In diesem Kapitel wird der strukturelle Aufbau des Projektes beschrieben. Dabei wird zunächst erläutert, welche methodischen Ansätze zur Analyse der Problemstellung und zur Entwicklung des Systemkonzeptes eingesetzt wurden. Anschließend wird dargestellt, wie das entwickelte Konzept in eine konkrete Systemarchitektur überführt wurde und welche zentralen Konzeptscheidungen im Verlauf des Projektes getroffen wurden. Abschließend werden die eingesetzten Technologien und Werkzeuge sowie mögliche Validierungs- und Messmethodiken vorgestellt, mit denen die Funktionalität und Robustheit des Systems bewertet werden können.

3.1 Eingesetzte Methoden

Zu Beginn des Projektes wurde die Problemstellung definiert. Ziel des Projektes war die Entwicklung eines Systems zur autonomen Indoor-Navigation mit dem Unitree Go2 Edu. Als Anwendungsszenario wurde ein autonomer Rundgang innerhalb des CobotsLabs der TH Köln betrachtet. Der Roboter sollte dabei in der Lage sein, den Raum zu kartieren, definierte Zielpunkte innerhalb dieses Raumes zu erreichen und potenziell wiederkehrende Routen zu übernehmen.

Auf Grundlage der Problemstellung wurden zunächst funktionale Anforderungen an das System abgeleitet. Der Roboter sollte in der Lage sein, mithilfe eines SLAM-Verfahrens eine Karte seiner Umgebung zu erstellen. Innerhalb dieser Karte sollte ein Zielpunkt definiert werden können, zu dem anschließend eine Pfadplanung durchgeführt wird. Der Roboter sollte darüber hinaus in der Lage sein, diesen Pfad autonom abzufahren. Zusätzlich wurde eine Not-Stopp-Funktion als sicherheitsrelevante Anforderung berücksichtigt.

Neben den funktionalen Anforderungen wurden auch nicht-funktionale Anforderungen definiert. Dazu gehörte insbesondere eine stabile Lokalisierung des Roboters innerhalb der Karte, eine effiziente Berechnung der Navigationspfade sowie eine kollisionsfreie Navigation innerhalb der Umgebung.

Für die Konzeptentwicklung wurde ein Baukasten-Ansatz gewählt. Dabei wurden verschiedene technische Komponenten identifiziert, die für die Umsetzung des Systems relevant sind. Dazu gehörten insbesondere die Sensorik, Verfahren zur Kartierung der Umgebung sowie Methoden zur Navigation und Pfadplanung. Im Bereich der Sensorik wurden mögliche Datenquellen wie LiDAR, Kamera oder IMU betrachtet. Für die Kartierung der Umgebung wurden verschiedene SLAM-Verfahren analysiert, darunter graphbasierte Verfahren, Scan-Matching-Ansätze sowie multisensorbasierte Verfahren, die mehrere Sensortypen kombinieren.

Darüber hinaus wurden verschiedene Algorithmen zur Pfadplanung untersucht. Dazu gehörten rasterbasierte Verfahren wie der A*-Algorithmus, der auf einer Rasterkarte den kürzesten Weg zwischen Start- und Zielpunkt berechnet, sowie stichprobenbasierte Verfahren wie RRT, die insbesondere in komplexeren Umgebungen eingesetzt werden können.

Die verschiedenen Konzeptansätze wurden anschließend hinsichtlich ihrer Hardware-Kompatibilität, ihrer Implementierungskomplexität sowie ihrer Robustheit bewertet. Auf dieser Grundlage wurde ein Konzept gewählt, das auf LiDAR-basierter Kartierung und darauf aufbauender Pfadplanung basiert.

3.2 Entwurfsphase

In diesem Kapitel wird die Überführung des zuvor entwickelten Konzepts in eine konkrete Systemarchitektur beschrieben. Dabei wird erläutert, wie die einzelnen Systemkomponenten strukturiert wurden und welche Rolle das Robotik-Framework ROS 2 innerhalb der Architektur einnimmt. Zudem wird der Datenfluss innerhalb des Systems dargestellt, angefangen bei der Sensordatenerfassung über die Kartierung der Umgebung bis hin zur Pfadplanung und Steuerung des Roboters. Ziel dieses Kapitels ist es, die technische Struktur des Systems sowie die Interaktion der einzelnen Komponenten nachvollziehbar darzustellen.

In der Entwurfsphase wurde das zuvor entwickelte Konzept in eine konkrete Systemarchitektur überführt. Als technologische Grundlage wurde das Robotik-Framework ROS 2 gewählt. ROS 2 stellt eine Sammlung von Bibliotheken, Tools und Kommunikationsmechanismen dar, die die Entwicklung komplexer Robotiksysteme erleichtern und eine modulare Softwarearchitektur ermöglichen.

Die grundlegende Datenpipeline des Systems wurde als Verarbeitungskette konzipiert. Zunächst erfasste der im Unitree Go2 verbaute Front-LiDAR Sensordaten der Umgebung. Diese Sensordaten wurden anschließend innerhalb der ROS-2-Architektur verarbeitet und an einen SLAM-Algorithmus weitergeleitet. Der SLAM-Algorithmus erzeugte aus den Sensordaten eine Karte der Umgebung und bestimmte gleichzeitig die aktuelle Position des Roboters innerhalb dieser Karte. Die erzeugte Karte wurde gespeichert und diente als Grundlage für die Navigation.

Auf Basis dieser Karte wurde anschließend ein Pfad zu einem definierten Zielpunkt berechnet. Hierbei kam ein Global Planner zum Einsatz, der den vollständigen Pfad vom Startpunkt zum Zielpunkt berechnete. Dieser Pfad wurde als Sequenz von Wegpunkten dargestellt. Ein Local Planner übernahm anschließend die Ausführung dieses Pfades und passte die Bewegung des Roboters kontinuierlich an mögliche Hindernisse oder Änderungen der Sensordaten an.

Zur Visualisierung und Analyse der Sensordaten sowie der generierten Karten wurde die Software RViz2 eingesetzt. Diese ermöglichte eine grafische Darstellung der Sensordaten, der erzeugten Karten sowie der geplanten Navigationspfade. Zusätzlich wurde das Unitree-SDK in die ROS-2-Architektur integriert, um eine direkte Ansteuerung der Hardware des Roboters zu ermöglichen. Die grundlegende Systemstruktur lässt sich somit vereinfacht als Datenfluss von LiDAR-Daten über SLAM zur Kartenerstellung und anschließend zur Pfadplanung beschreiben.

3.3 Konzeptentscheidungen

Dieses Kapitel beschreibt die zentralen technischen Entscheidungen, die im Verlauf der Konzeptentwicklung getroffen wurden. Dazu gehört insbesondere die Wahl der verwendeten Sensorik sowie die Entscheidung, ausschließlich die serienmäßig im Unitree Go2 verbaute Hardware zu nutzen. Darüber hinaus wird erläutert, wie das Projekt mithilfe von Minimum Viable Products (MVPs) strukturiert wurde, um eine schrittweise Entwicklung der Kernfunktionen zu

ermöglichen. Zusätzlich werden mögliche Erweiterungen und weiterführende Meilensteine beschrieben, die über die grundlegenden Projektziele hinausgehen.

Im Rahmen der Konzeptentwicklung wurden mehrere grundlegende Designentscheidungen getroffen. Eine zentrale Entscheidung bestand darin, ausschließlich die serienmäßig im Unitree Go2 verbaute Sensorik zu verwenden, um die Kosten der Education-Version unverändert zu halten und die bestehende Funktionslücke ausschließlich softwareseitig zu lösen. Viele bestehende Implementierungen für autonome Navigation setzen zusätzliche externe 3D-LiDAR-Systeme ein, deren Kosten typischerweise zwischen etwa 3000 und 6000 Euro liegen.

Der Fokus lag stattdessen auf der Nutzung des bereits integrierten Front-LiDAR-Sensors des Roboters. Ziel dieser Entscheidung war es, eine möglichst kostengünstige und reproduzierbare Basislösung zu entwickeln, die ohne zusätzliche Hardware auskommt und dennoch eine grundlegende autonome Navigation ermöglicht. Das Projekt sollte damit als Machbarkeitsdemonstrator für autonome Navigation mit minimaler Sensorik dienen.

Zur Strukturierung des Projektverlaufs wurden sogenannte Minimum Viable Products (MVPs) definiert. Diese Zwischenziele sollten sicherstellen, dass die wichtigsten Funktionen des Systems schrittweise entwickelt und überprüft werden konnten.

Das erste MVP bestand darin, dass der Roboter mithilfe seines Front-LiDARs eine Karte seiner Umgebung erstellen kann. Dabei sollte die Umgebung gescannt, eine Karte erzeugt und anschließend gespeichert werden. Das zweite MVP bestand darin, dass auf Grundlage dieser gespeicherten Karte eine Pfadplanung durchgeführt werden kann. Dazu sollte ein Zielpunkt in der Karte definiert werden, zu dem anschließend ein kollisionsfreier Pfad berechnet wird.

Darüber hinaus wurden weitere mögliche Entwicklungsschritte definiert, die über die grundlegenden MVPs hinausgehen und als sogenannte „Nice-to-Have“-Meilensteine betrachtet werden können. Dazu gehören beispielsweise die autonome Bewegung des Roboters entlang des berechneten Pfades, eine Live-Hinderniserkennung mit automatischen Ausweichbewegungen sowie eine stabile Echtzeit-Lokalisierung während der Navigation. Weitere mögliche Erweiterungen wären die Integration eines Checkpoint-Systems für Sicherheitsrundgänge, die Veröffentlichung des Projektes als Open-Source-Repository sowie die Erweiterung des Systems durch zusätzliche Sensorik wie Kameras oder weitere LiDAR-Sensoren.

3.4 Technologien und Werkzeuge

In diesem Kapitel werden die im Projekt eingesetzten Technologien und Werkzeuge vorgestellt. Dazu gehören sowohl die verwendeten Softwareplattformen und Frameworks als auch die eingesetzte Hardware. Darüber hinaus werden die Entwicklungsumgebung sowie die verwendeten Tools zur Implementierung des Systems beschrieben. Abschließend werden mögliche Validierungs- und Messmethodiken erläutert, mit denen die Genauigkeit und Robustheit des entwickelten Systems bewertet werden können.

Für die Umsetzung des Projektes wurden verschiedene Technologien und Werkzeuge eingesetzt. Als zentrale Softwareplattform wurde das Robotik-Framework ROS 2 verwendet. Dieses fungiert als Middleware zwischen den einzelnen Systemkomponenten und ermöglicht eine modulare Entwicklung von Robotikanwendungen. Die Kommunikation zwischen den einzelnen Softwarekomponenten und dem Roboter erfolgte über eine Datenkabelverbindung zwischen dem PC und dem Unitree Go2.

Für die Kartierung der Umgebung wurde ein LiDAR-basierter SLAM-Ansatz verwendet. Zur Visualisierung der Sensordaten, der erzeugten Punktwolken sowie der generierten Karten wurde die Software RViz2 eingesetzt.

Auf Hardwareseite wurde der Unitree Go2 Edu verwendet. Dieser verfügt über einen integrierten Front-LiDAR-Sensor sowie eine IMU-Sensorik, die zusätzliche Bewegungsinformationen liefert.

Als Entwicklungsumgebung wurde ein Linux-basiertes System mit Ubuntu verwendet. Zusätzlich wurde eine Docker-basierte Entwicklungsumgebung eingerichtet, um eine reproduzierbare Softwareumgebung zu gewährleisten. Für die Versionsverwaltung des Projektes wurde GitHub eingesetzt.

Neben der technischen Umsetzung wurden bereits im Projektkonzept mögliche Validierungs- und Messmethodiken definiert, mit denen die Robustheit und Genauigkeit des Systems bewertet werden können.

Zur Bewertung der Lokalisierungsgenauigkeit können beispielsweise die Metriken Absolute Trajectory Error (ATE) und Relative Pose Error (RPE) verwendet werden. Der ATE beschreibt die Abweichung zwischen der geschätzten Trajektorie des Roboters und einer Referenztrajektorie über den gesamten Bewegungsverlauf hinweg und ermöglicht damit eine Analyse von Drift zwischen Start- und Endpunkt. Der RPE betrachtet hingegen lokale Positionsabweichungen zwischen aufeinanderfolgenden Trajektorienpunkten und ermöglicht eine Bewertung der kurzfristigen Positionsgenauigkeit.

Zur Bewertung der Kartengenauigkeit können weitere Metriken verwendet werden. Dazu gehört beispielsweise der sogenannte Scale Error, der das Verhältnis zwischen gemessenen und tatsächlichen Distanzen innerhalb der Karte beschreibt. Eine weitere Kennzahl ist die Coverage, also der prozentuale Anteil des Raumes, der erfolgreich kartiert wurde. Zusätzlich kann die Positionsabweichung einzelner Objekte innerhalb der Karte gemessen werden.

Für die Auswertung der Trajektorien kann beispielsweise die Python-Bibliothek *evo* (Evaluation of Odometry) verwendet werden. Diese ermöglicht eine quantitative Analyse von Trajektorien durch zeitliche Synchronisation der Daten, räumliche Ausrichtung mittels Umeyama-Algorithmus sowie die Berechnung euklidischer Positionsabweichungen.

Neben der Kartierung wurde auch eine mögliche Testmethodik für das Pathfinding definiert. Hierfür kann ein Labyrinth-Testlayout erstellt werden, bei dem Hindernisse wie Kartons oder Möbelstücke zwischen Start- und Zielpunkt platziert werden. Anschließend wird überprüft, ob der berechnete Pfad kollisionsfrei ist, ob ein definierter Sicherheitsabstand von mindestens 20 cm eingehalten wird und wie lange die Pfadplanung benötigt. Als weiteres Testverfahren kann ein Robustheitstest durchgeführt werden, bei dem ein Zielpunkt bewusst nicht erreichbar ist. In diesem Fall wird überprüft, ob das System korrekt erkennt, dass kein gültiger Pfad existiert und eine entsprechende Fehlermeldung ausgibt.

4 Darstellung der Ergebnisse

In diesem Kapitel wird die technische Umsetzung der SLAM-Implementierung auf dem Unitree Go2 beschrieben. Dabei werden die gewählte Software-Architektur, der Funktionsumfang sowie die im Projektverlauf erzielten (messbaren) Resultate detailliert dargestellt.

4.1 Umgesetzte Lösung

Die technische Basis des Projekts bildet ein Ubuntu 22.04 System in Kombination mit ROS 2. Um eine reproduzierbare Entwicklungsumgebung zu gewährleisten, wurde die gesamte Applikationslogik in einem Docker-Container realisiert. Die Hardware-Anbindung erfolgte über eine kabelgebundene Ethernet-Verbindung zum Onboard-Rechner des Unitree Go2, wobei die Punktwolkendaten über das offizielle Unitree SDK abgegriffen wurden.

Während das initiale Konzept eine Multi-Sensor-Fusion via Fast-LIO vorsah, beschränkt sich die finale Umsetzung aufgrund technischer Hürden bei der Datenintegration auf KISS-ICP unter ausschließlicher Nutzung des Front-LiDAR-Sensors. Dass die Datenakquise und die Anbindung an die ROS-Middleware technisch erfolgreich umgesetzt wurden, belegt die Visualisierung der Rohdaten (Abbildung 4.1).

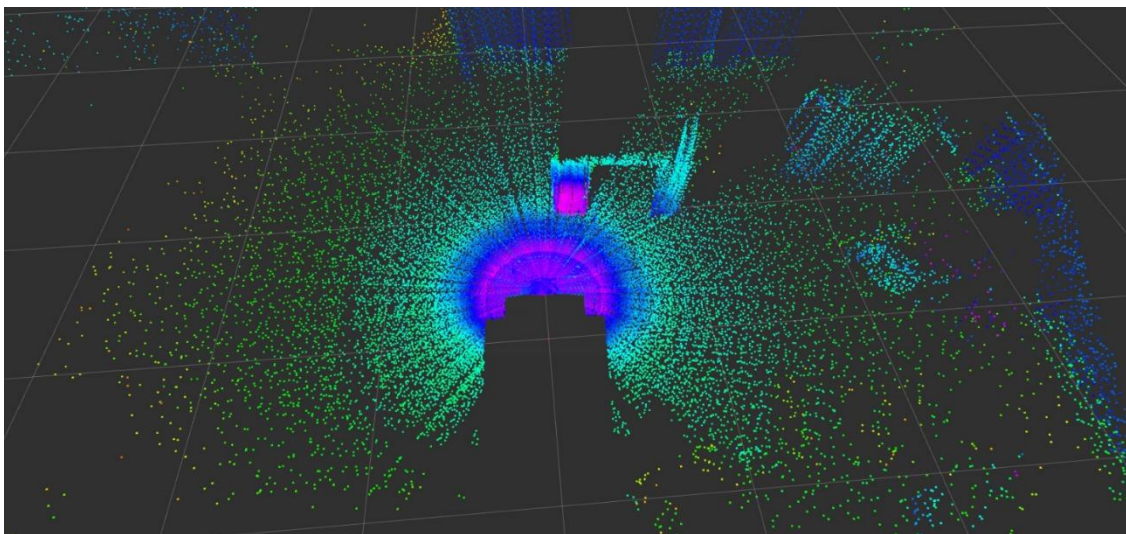


Abbildung 1: Pointcloud Daten des Frontlidars (eigene Darstellung)

4.2 Funktionsumfang

Der realisierte Funktionsumfang konzentriert sich primär auf die Pipeline zur Datenakquise und deren Visualisierung. Ein zentraler Bestandteil ist die Daten-Pipeline, welche den Abruf der Roh-Punktwolken vom LiDAR-Sensor via SDK und die Weitergabe an den ROS 2-Node von KISS-ICP steuert. Zur Überprüfung der Ergebnisse wurde eine Live-Visualisierung mittels RViz2 integriert, die die Darstellung der aktuellen Punktwolke sowie des Mapping-Prozesses in Echtzeit erlaubt.

Darüber hinaus wurde versucht, über die Transformationen (TF) kontinuierlich Odometrie-Daten zur Bestimmung der Roboterpose im Raum zu berechnen. Ein wesentliches Ziel des Projekts war zudem die Implementierung von Pathfinding-Algorithmen, um autonom festgelegte Routen abzulaufen und Zielpunkte anzufahren. Aufgrund der Instabilität des SLAM-Algorithmus konnten jedoch weiterführende Funktionen wie der Loop Closure (Schleifenschluss zur Kartenkorrektur) oder die Erstellung eines statischen 2D-Occupancy-Grids nicht zielführend implementiert werden. Da keine konsistente Karte als Grundlage vorlag, konnte auch das darauf aufbauende Pathfinding nicht umgesetzt werden, da das System zu keinem Zeitpunkt eine verlässliche Lokalisierung innerhalb einer stabilen Umgebungskarte besaß.

4.3 Technische Resultate

Die technische Auswertung zeigt, dass mit der gewählten Konfiguration keine konsistente Kartierung des Umfelds erreicht werden konnte. Das Hauptproblem stellte die Instabilität des Koordinatenursprungs (Map-Frame) dar, der bereits im stationären Zustand des Roboters eine starke Drift und unkontrollierte Sprünge aufwies.

Die in RViz2 generierten Karten verdeutlichen das Ausmaß dieser Instabilität (Abbildung 4.3). Da der Algorithmus die aufeinanderfolgenden Scans nicht korrekt registrieren konnte, entstanden inkonsistente Überlagerungen, die keine verwertbaren Raumstrukturen wie Wände oder Hindernisse erkennen lassen.

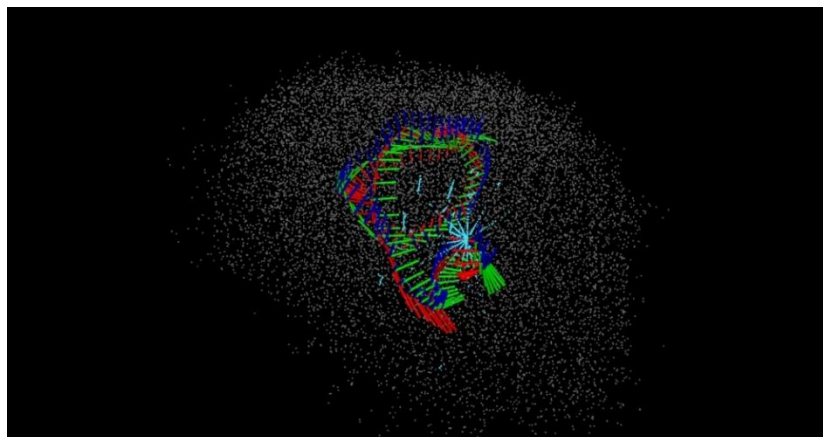


Abbildung 2: Mit RViz2 erstellte Map (eigene Darstellung)

5 Auswertung und Diskussion der Ergebnisse

In diesem Kapitel werden die im Projektverlauf erzielten Ergebnisse analysiert und dem ursprünglichen Zielbild gegenübergestellt. Ziel des Projektes war die Realisierung einer autonomen Indoor-Navigation mit dem Unitree Go2 Edu auf Basis einer LiDAR-basierten Kartierung der Umgebung. Während die technische Infrastruktur erfolgreich aufgebaut werden konnte, konnten die funktionalen Kernziele der autonomen Navigation aufgrund erheblicher Instabilitäten im SLAM-Prozess nicht erreicht werden.

Positiv hervorzuheben ist zunächst die erfolgreiche Realisierung der grundlegenden Kommunikationsstruktur zwischen der Hardware des Unitree Go2 und der ROS-2-Umgebung. Die Integration des offiziellen Unitree-SDKs in eine containerisierte Entwicklungsumgebung auf Basis von Docker ermöglichte einen stabilen Zugriff auf die LiDAR-Rohdaten des Roboters. Die Visualisierung der Sensordaten in RViz2 bestätigte, dass die vom Front-LiDAR erzeugten Punktwolken korrekt im ROS-System verarbeitet und dargestellt werden konnten. Damit wurde sichergestellt, dass die Sensorinformationen grundsätzlich für weiterführende Algorithmen zur Verfügung standen.

Die Auswertung des gewählten SLAM-Verfahrens (KISS-ICP) zeigte jedoch erhebliche Einschränkungen bei der Erstellung konsistenter Karten. Anstelle einer stabilen Repräsentation der Raumstruktur wurden in der praktischen Anwendung lediglich inkonsistente Überlagerungen von Punktwolken erzeugt. Ein zentrales Problem stellte dabei die Instabilität des sogenannten Map-Frames dar. Bereits im stationären Zustand des Roboters zeigte der Koordinatenursprung eine deutliche Drift sowie sprunghafte Positionsänderungen. Dies führte dazu, dass der Roboter seine eigene Position innerhalb der Karte selbst im Stillstand nicht stabil bestimmen konnte.

Darüber hinaus zeigte sich, dass aufeinanderfolgende LiDAR-Scans vom Algorithmus nicht zuverlässig registriert werden konnten. Dadurch entstanden sogenannte „Geisterwände“, bei denen sich identische Raumstrukturen mehrfach und versetzt überlagerten. Eine klare und konsistente Darstellung der Raumgeometrie konnte unter diesen Bedingungen nicht erzeugt werden. Da eine zuverlässige Karte jedoch die grundlegende Voraussetzung für eine autonome Navigation darstellt, konnte auf dieser Basis keine funktionierende Navigationsstrategie umgesetzt werden.

Diese Instabilitäten hatten direkte Auswirkungen auf die geplante Pfadplanung im Rahmen des zweiten Minimum Viable Products (MVP 2). Da kein stabiles 2D-Occupancy-Grid erstellt werden konnte, verfügte das Navigationsframework Nav2 über keine zuverlässige Datengrundlage zur Berechnung kollisionsfreier Pfade. Somit war weder die sichere Ansteuerung definierter Zielpunkte noch die Umsetzung autonomer Navigationsbewegungen möglich. Das Projekt verblieb daher auf der Stufe eines experimentellen Aufbaus der Datenpipeline, ohne dass die angestrebten Autonomieziele vollständig realisiert werden konnten.

6 Fehlerbetrachtung und Reflexion der eigenen Vorgehensweise

Die Analyse des Projektverlaufs zeigt, dass das Scheitern der autonomen Navigation auf eine Kombination aus technischen Einschränkungen der eingesetzten Hardware sowie methodischen Herausforderungen bei der Softwareintegration zurückzuführen ist.

6.1 Technische Limitationen

Eine wesentliche technische Einschränkung ergab sich aus der geometrischen Anordnung des Sensors innerhalb der Roboterplattform. Das Sichtfeld des Front-LiDARs wird nach hinten durch den Körper des Roboters teilweise verdeckt. Diese Okklusion reduziert die Anzahl verfügbarer geometrischer Merkmale, die für das Scan-Matching genutzt werden können. Insbesondere bei Rotationsbewegungen des Roboters führt diese Einschränkung dazu, dass weniger stabile Referenzpunkte für die Registrierung aufeinanderfolgender Scans zur Verfügung stehen.

Ein weiterer entscheidender Faktor war das Fehlen einer stabilen Sensorfusion. Die ursprüngliche Systemarchitektur sah die Nutzung einer LiDAR-Inertial-Odometrie auf Basis des Verfahrens FAST-LIO2 vor. Aufgrund technischer Schwierigkeiten bei der Integration der IMU-Daten über das Unitree-SDK musste jedoch auf einen LiDAR-only-Ansatz (KISS-ICP) ausgewichen werden. Ohne die stabilisierende Wirkung der IMU reagierte das System deutlich empfindlicher auf mechanische Vibrationen sowie auf ruckartige Bewegungen des Roboters während der Fortbewegung. Diese Einflüsse wirkten sich unmittelbar negativ auf die Stabilität der Trajektorien-schätzung aus.

6.2 Methodische Einschränkungen

Neben den technischen Faktoren spielten auch methodische Aspekte eine Rolle im Projektverlauf. Die im Vorfeld durchgeführte Befragung zur Validierung der Problemstellung basierte lediglich auf einer sehr kleinen Stichprobe mit drei Teilnehmenden. Diese Befragung diente daher ausschließlich der explorativen Plausibilisierung des Anwendungsszenarios und besitzt aufgrund der geringen Stichprobengröße keine statistische Aussagekraft für die Sicherheitsbranche insgesamt.

Darüber hinaus zeigte sich im Projektverlauf, dass die Komplexität der Datenintegration sowie der Konfiguration der Middleware initial unterschätzt wurde. Insbesondere die Integration herstellerspezifischer Software-Schnittstellen in das ROS-2-Framework erwies sich als aufwändiger als ursprünglich angenommen. Diese zusätzlichen Implementierungsaufwände führten dazu, dass im späteren Projektverlauf weniger Zeit für die iterative Optimierung der SLAM-Parameter zur Verfügung stand.

6.3 Ressourcenbedingte Einschränkungen

Neben technischen und methodischen Faktoren wurde der Projektverlauf auch durch die verfügbaren Ressourcen beeinflusst. Die zeitliche Begrenzung des Projektes auf die Dauer eines Semesters sowie die personellen Kapazitäten der Projektgruppe setzten dem Umfang der Untersuchungen klare Grenzen.

Im Rückblick lässt sich feststellen, dass die technologische Komplexität des Vorhabens insbesondere im Hinblick auf die Integration herstellerspezifischer Softwarekomponenten unterschätzt wurde. Die initiale Zeitplanung berücksichtigte nicht in ausreichendem Maße den Aufwand für die Fehlersuche bei Inkompatibilitäten zwischen dem Unitree-SDK und den eingesetzten SLAM-Bibliotheken. Die dadurch entstandenen Verzögerungen in der Implementierungsphase führten dazu, dass das Zeitfenster für die iterative Optimierung der SLAM-Parameter sowie für die Durchführung systematischer Testreihen, wie beispielsweise des geplanten Labyrinth-Tests, deutlich reduziert wurde.

6.4 Learning Outcomes

Trotz der nicht erreichten funktionalen Navigationsziele konnten im Rahmen des Projektes wesentliche Lernergebnisse erzielt werden. Auf technischer Ebene erlangte die Projektgruppe

vertiefte Kenntnisse im Umgang mit containerisierten Entwicklungsumgebungen für Robotiksysteme sowie in der Konfiguration komplexer Softwarearchitekturen auf Basis von ROS 2. Insbesondere die Integration verschiedener Softwarekomponenten innerhalb eines Docker-basierten Robotik-Stacks stellte einen wichtigen Kompetenzzuwachs dar.

Darüber hinaus wurden fundierte praktische Erfahrungen in der Verarbeitung und Visualisierung von Punktwolken gesammelt. Die intensive Auseinandersetzung mit ICP-basierten SLAM-Verfahren sowie den zugrunde liegenden mathematischen Konzepten der Punktwolkenregistrierung führte zu einem vertieften Verständnis moderner Verfahren der robotischen Wahrnehmung.

Neben den technischen Kompetenzen konnten auch methodische Erkenntnisse im Bereich Projektmanagement gewonnen werden. Insbesondere wurde deutlich, wie wichtig eine realistische Aufwandsschätzung für Forschungs- und Entwicklungsprojekte ist. Darüber hinaus zeigte sich die Bedeutung eines flexiblen und agilen Risikomanagements, insbesondere in Situationen, in denen geplante technische Ansätze aufgrund unvorhergesehener Integrationsprobleme kurzfristig angepasst werden müssen.

7 Fazit

Das vorliegende Projekt verfolgte das Ziel, eine kosteneffiziente und technisch belastbare Lösung für autonome Sicherheitsrundgänge an der TH Köln zu entwickeln, indem ausschließlich die serienmäßige Sensorik des Unitree Go2 genutzt wird. Die technologische Vision sah ein System vor, das ausgewählte Innenbereiche selbstständig kartiert und vordefinierte Zielpunkte entlang sicherheitsrelevanter Routen autonom ansteuert. Zur schrittweisen Validierung wurde das Vorhaben in zwei Kernziele unterteilt: die Erstellung einer SLAM-basierten Karte (MVP 1) sowie die Durchführung einer kollisionsfreien Pfadplanung (MVP 2).

Im Rahmen der Projektdurchführung konnte die grundlegende Daten-Pipeline erfolgreich etabliert werden. Die Implementierung einer modularen Softwarearchitektur auf Basis von ROS 2 innerhalb einer Docker-Umgebung ermöglichte eine reproduzierbare Anbindung an die Hardware der quadrupeden Roboterplattform. Die erfolgreiche Akquise und Visualisierung der LiDAR-Punktwolken über das Unitree-SDK belegen die prinzipielle Funktionsfähigkeit der Kommunikationswege zwischen Sensorik und Middleware.

Dennoch muss das Projektergebnis im Hinblick auf die angestrebte Autonomie kritisch bewertet werden, da das übergeordnete Ziel einer stabilen Indoor-Navigation nicht vollständig erreicht wurde. Die technische Auswertung verdeutlichte, dass die ausschließliche Nutzung des integrierten Front-LiDARs mittels KISS-ICP unter den gegebenen Rahmenbedingungen keine konsistente Kartierung ermöglichte. Aufgrund massiver Instabilitäten im Map-Frame und signifikanter Registrierungsfehler der Punktwolken konnte kein belastbares Occupancy-Grid erzeugt werden. Da keine verlässliche Lokalisierung innerhalb einer stabilen Umgebungskarte vorlag, war die Umsetzung einer autonomen Pfadplanung technisch nicht realisierbar.

Für zukünftige Entwicklungszyklen ergeben sich daraus wesentliche Handlungsempfehlungen zur Optimierung des Systems. Eine zentrale Voraussetzung für die Stabilisierung der Odometrie ist die Implementierung einer eng gekoppelten Sensor-Fusion, welche die LiDAR-Daten mit den Werten der Inertialsensorik (IMU) verknüpft. Darüber hinaus sollte die Installation zusätzlicher 3D-LiDAR-Hardware auf dem Rücken der Roboterplattform evaluiert werden, um die physikalisch bedingten Verdeckungen des Sichtfeldes durch den Korpus zu kompensieren und die Merkmalsdichte für den SLAM-Algorithmus zu erhöhen. Die verstärkte Nutzung von Simulationsumgebungen zur Parameterkalibrierung sowie die Durchführung systematischer Testreihen unter kontrollierten Bedingungen bieten weiteres Potenzial, um die Robustheit der Navigation langfristig zu steigern. Zusammenfassend liefert die Arbeit wichtige Erkenntnisse über die systemischen Anforderungen an mobile Robotiksysteme im Sicherheitskontext und bildet eine fundierte wissenschaftliche Basis für weiterführende Forschungsarbeiten an der TH Köln.

Literaturverzeichnis

- [1] "Unitree Shop," *Unitree Robotics*, o. D. [Online]. Verfügbar unter: <https://shop.unitree.com/>. [Abgerufen am: 06. März 2026].
- [2] "Unitree Shop," *Unitree Robotics*, o. D. [Online]. Verfügbar unter: <https://shop.unitree.com/>. [Abgerufen am: 06. März 2026].
- [3] "Fachkräftemangel in der Sicherheitsbranche: Ursachen und Lösungsansätze," *KWS-Sicherheit*, o. D. [Online]. Verfügbar unter: <https://kws-sicherheit.de/fachkraefte-mangel-in-der-sicherheitsbranche-ursachen-und-loesungsansaetze/>. [Abgerufen am: 06. März 2026].
- [4] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Comput. Intell. Robot. Autom.*, 1997.
- [5] H. Gonzalez-Banos and J.-C. Latombe, "Navigation strategies for exploring indoor environments," *Int. J. Robot. Res.*, 2002.
- [6] C. Cadena *et al.*, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, 2016.
- [7] Y. Zhu *et al.*, "Target-driven visual navigation in indoor scenes," *arXiv preprint arXiv:1609.05143*, Sep. 2016.
- [8] S. Thrun, "Probabilistic algorithms in robotics," Carnegie Mellon University, 2000.
- [9] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," 2021.
- [10] T. Guadagnino, B. Mersch, S. Gupta, I. Vizzo, G. Grisetti, and C. Stachniss, "KISS-SLAM: A simple, robust, and accurate 3D LiDAR SLAM system with enhanced generalization capabilities," 2025.
- [11] A. Waga *et al.*, "A survey on autonomous navigation for mobile robots: From traditional techniques to deep learning and large language models," 2025.
- [12] Navigation2 Project, "Nav2 Documentation," ROS 2 Navigation Framework.
- [13] Y. Tang *et al.*, "Path planning trends for autonomous mobile robot navigation: A review," *Sensors*, vol. 25, no. 4, 2025.
- [14] Open Robotics, "Basic Concepts," ROS 2 Documentation. [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Basic.html>.
- [15] Unitree Robotics, "Go2 Product Page." [Online]. Available: <https://www.unitree.com/go2>.
- [16] Unitree Robotics, "Unitree Robotics Developer Documentation," [Online]. Available for LiDAR service: https://support.unitree.com/home/en/developer/LiDAR_service. ROS2 service: https://support.unitree.com/home/en/developer/ROS2_service. SLAM and Navigation service: https://support.unitree.com/home/en/developer/SLAM%20and%20Navigation_service. Accessed: Mar. 6, 2026.
- [17] H. Yin *et al.*, "A survey on global LiDAR localization," preprint, 2024.

Anhangsverzeichnis

Abbildungsverzeichnis

Abbildung 1: Pointcloud Daten des Frontlidars (eigene Darstellung)	22
Abbildung 2: Mit RViz2 erstellte Map (eigene Darstellung)	23